

11. More About Inheritance

Example 11.1

```
class Animal {
    protected String name;

    public Animal(String name) {
        this.name = name;
    }

    public String speak() {
        return "Animal sound";
    }
}

class Dog extends Animal {

    public Dog(String name) {
        super(name);
    }

    @Override
    public String speak() {
        return "Dog barks";
    }
}

public class Main {
    public static void main(String[] args) {
        Animal a = new Dog("Buddy");
        System.out.println(a.speak());
    }
}
```

This example shows:

- inheritance
- protected field
- method overriding
- static type vs dynamic type
- dynamic method lookup
- method polymorphism

11.1. Static Type and Dynamic Type

Definition

- **Static type:** the type of a variable at compile time
- **Dynamic type:** the real type of the object at runtime

Features

- Static type is decided by the variable declaration
 - Dynamic type is decided by the object created with `new`
 - They can be different
-

Example

```
Animal a = new Dog("Buddy");
```

- static type of `a` is `Animal`
 - dynamic type of `a` is `Dog`
-

Remark

- Java checks **method existence** using static type
 - Java chooses **method implementation** using dynamic type
-

11.2. Method Overriding

Definition

- **Overriding** means a subclass provides its own version of a superclass method
-

Features

- Method name must be the same
 - Parameter list must be the same
 - Return type must be the same or a subtype
 - Access level cannot be more restrictive
-

Example

```
@Override
public String speak() {
    return "Dog barks";
}
```

- `Dog` overrides `Animal.speak()`

Remark

- `@Override` helps catch errors
- Overriding works only with **instance methods**

11.3. Redefinition vs Overriding

Definition

- **Overriding**: same method signature in subclass
- **Redefinition**: a broader term, may include static methods

Features

- Overriding uses **dynamic binding**
- Static methods are **redefined**, not overridden

Example

```
// Static methods are redefined, not overridden
```

- Static methods depend on static type

Remark

- Exams often ask this difference

11.4. Method Polymorphism

Definition

- **Method polymorphism** means the same method call behaves differently for different objects

Features

- Based on inheritance
- Based on overriding
- Uses dynamic type

Example

```
Animal a = new Dog("Buddy");  
a.speak();
```

- Call is the same
- Result depends on the object type

Remark

- Polymorphism improves extensibility
- No `if-else` needed

11.5. Method Lookup

Definition

- **Method lookup** is the process of finding the correct method to run

Features

- Starts from the dynamic class
- Moves up the inheritance tree
- Stops at the first match

Example

```
a.speak();
```

- JVM looks for `speak()` in `Dog`
- If not found, looks in `Animal`

Remark

- Method lookup happens at runtime

11.6. Method Dispatch

Definition

- **Method dispatch** is the act of calling the selected method
-

Features

11.6.1 Dynamic Dispatch

- Used for overridden instance methods
- Based on dynamic type

11.6.2 Static Dispatch

- Used for static methods
 - Based on static type
-

Example

```
Animal a = new Dog("Buddy");  
a.speak();
```

- Uses **dynamic dispatch**
-

Remark

- Dynamic dispatch enables polymorphism
-

11.7. Protected Access Modifier

Definition

- `protected` allows access in the same package or subclasses
-

Features

11.7.1 Access Scope

- Same class
- Same package
- Subclasses

11.7.2 Common Usage

- Used for inheritance support
-

Example

```
protected String name;
```

- Subclass `Dog` can access `name`
-

Remark

- `protected` is more open than `private`
 - Used often in superclass design
-

Chapter 11 Key Exam Summary

- Static type $\neq$ dynamic type
- Overriding depends on dynamic type
- Method calls use dynamic method lookup
- Polymorphism works only with instance methods
- `protected` supports inheritance
- Static methods do **not** use dynamic dispatch